

7 — Reactive object: reads track, writes trigger, through plain dots

PROXY OBSERVER REACTIVITY

Q. With `reactive({ claps: 0 })`, you never call `subscribe`. So why does `article.claps = 2` re-run the effect that read it?

ANSWER. Because the object is wrapped in a proxy whose traps are wired to the track-and-trigger engine. Reading `article.claps` runs the `get` trap, which records the active effect. Writing `article.claps = 2` runs the `set` trap, which re-runs every recorded effect. You wrote plain dot syntax; the proxy did the bookkeeping.

WHY IT WORKS. The pieces from the last two cards are joined here. The proxy intercepts every property, even ones added later. The `get` trap uses that interception to *track* whoever is reading. The `set` trap uses it to *trigger* everyone who ever read. The surface stays clean — `article.claps`, no method calls — and underneath, an entire dependency graph keeps itself in sync.

```
let activeEffect = null; // who's
reading right now?
function reactive(target) {
  const deps = new Set();
  return new Proxy(target, { // wrap in a
    proxy
      get(obj, key) {
        if (activeEffect) deps.add(activeEffect); //
        track on read
        return obj[key];
      },
      set(obj, key, value) {
        obj[key] = value;
        deps.forEach(run => run()); // trigger on
        write
        return true;
      }
    });
}
function effect(fn) {
  activeEffect = fn;
  fn();
}
```

examples/reactive.js

```
    activeEffect = null;
  }
  const article = reactive({ claps: 0 });
  effect(() => console.log("claps:", article.claps));
  // "claps: 0"
  article.claps = 1;
  // "claps: 1"
  article.claps = 2;
  // "claps: 2"
```

ONE SURFACE, TWO FACES

article.claps = 2

CLEAN SYNTAX · NO SUBSCRIBE CALL



get TRAP

on read

TRACKS THE READER

set TRAP

on write

TRIGGERS READERS

- ① **Summary.** A reactive object hides the track-and-trigger engine behind ordinary dot-notation: writes trigger, reads track, all through proxy traps. This is almost exactly what a framework hands you, and what Svelte calls a rune.